INSTITUTO TECNOLÓGICO DE LAS AMÉRICAS (ITLA)

Introducción a la Programación

Capítulo IX Arreglos y Vectores

Sistema de Control de Calificaciones

Estudiante: Amauri Feliz Valenzuela

Profesor: Jimmy Gómez

Modalidad: trabajo individual

Lenguaje: Python | Entorno previsto: Visual Studio Code

10 de octubre de 2026

Repositorio público disponible | Versión para revisión: capturas auténticas pendientes

Índice

Introducción	3
Objetivos	3
Concepto de arreglo	3
Índices y subíndices	4
Vectores y características	4
Descripción del proyecto	5
Desarrollo del programa	5
Explicación del código	6
Repositorio de GitHub	7
Evidencias de ejecución	7
Casos de prueba	8
Dificultades encontradas y soluciones	9
Conclusión	9
Fuentes consultadas	9

Introducción

Organizar varias calificaciones permite consultar resultados sin mantener una variable diferente para cada estudiante. Este trabajo estudia los arreglos, los índices y los vectores mediante un sistema de consola en Python. El ejemplo reúne cinco nombres y cinco notas, ofrece un menú y comprueba que las entradas se encuentren dentro de los límites establecidos.

Las explicaciones corresponden al código entregado y las pruebas se apoyan en transcripciones reales. La ejecución se realizó con Python 3.12.14 desde una consola automatizada en Windows. Visual Studio Code es el entorno solicitado para abrir y explicar el proyecto; no se afirma que haya sido utilizado durante las ejecuciones registradas. Las capturas de su interfaz siguen pendientes.

Objetivos

El objetivo general es demostrar cómo almacenar y procesar información con listas que representan vectores unidimensionales.

Los objetivos específicos son relacionar nombres y notas por sus índices; consultar y modificar posiciones; recorrer listas con ciclos; calcular resultados; aplicar condiciones para clasificar estudiantes; validar entradas y comprobar el programa con casos reproducibles.

Concepto de arreglo

Un arreglo reúne varios elementos bajo un mismo nombre y permite consultar sus posiciones. Se utiliza para manejar conjuntos como nombres, temperaturas o calificaciones. En lugar de repetir una operación para muchas variables separadas, se puede recorrer el conjunto. Esto facilita organizar los datos y evita escribir instrucciones similares una y otra vez.

Una variable individual como nota = 85 guarda una sola calificación. Una lista como la siguiente reúne cinco:

```
nota = 85
calificaciones = [85, 67, 92, 74, 58]
print(nota)
print(calificaciones[0])
```

Ambas instrucciones de impresión muestran 85, pero la segunda consulta una posición dentro de una colección. Una lista también puede almacenar texto o valores lógicos. Para este ejercicio se separan los nombres, que son cadenas, y las notas, que son enteros.

Conviene distinguir términos: muchos lenguajes tienen arreglos de tamaño fijo y elementos del mismo tipo. Las listas de Python pueden cambiar de tamaño y admitir tipos diferentes. Aquí se utilizan como una representación práctica de arreglos, sin afirmar que todas sus propiedades sean iguales. Las ventajas se observan al recorrer las cinco notas con una sola secuencia de instrucciones.

Índices y subíndices

El índice identifica una posición. En la expresión nombres[2], el número dentro de los corchetes actúa como subíndice y permite acceder al tercer elemento. En este contexto se usan ambos términos para hablar del acceso a una posición.

Python numera las posiciones desde cero por convención. Una forma de comprenderlo es contar cuántas posiciones se avanza desde el comienzo: para el primer elemento no se avanza ninguna. No todos los lenguajes empiezan en cero. Una lista de cuatro elementos tiene longitud 4 y último índice no negativo 3; en general, ese último índice es la longitud menos uno cuando la lista no está vacía.

```
nombres = ["Ana", "Luis", "Carlos", "Maria"]
print(nombres[0])
print(nombres[2])
nombres[1] = "Pedro"
```

La primera impresión muestra Ana y la segunda Carlos. La última instrucción cambia Luis por Pedro y no imprime nada por sí misma. Después, la lista queda como ['Ana', 'Pedro', 'Carlos', 'Maria']. Consultar nombres[4] produce IndexError, porque esa posición no existe. El programa valida el índice antes de acceder a las listas para evitar ese error.

Python también admite índices negativos; por ejemplo, -1 identifica el último elemento. El menú de este proyecto acepta únicamente índices de 0 a 4 para mantener una numeración clara y evitar confundir una entrada negativa con una consulta válida.

Vectores y características

En programación introductoria, un vector es un arreglo unidimensional. Unidimensional significa que se necesita un solo índice para ubicar un elemento. No es una tabla de filas y columnas. En este trabajo cada lista es una secuencia ordenada, accesible por posición, recorrible y modificable.

El ciclo for permite visitar los elementos. El siguiente ejemplo imprime 85, 67 y 92, cada uno en una línea:

```
notas = [85, 67, 92]
for nota in notas:
    print(nota)
```

También se puede recorrer con while, controlando una posición y aumentándola después de cada consulta:

```
indice = 0
while indice < len(notas):
    print(notas[indice])
    indice = indice + 1
```

Este ejemplo produce los mismos valores. La condición evita acceder al índice 3, que no existe. Aumentar el índice es necesario para que el ciclo termine. En el programa, for recorre las listas y while mantiene el menú activo y repite las preguntas cuando hay una entrada inválida.

Descripción del proyecto

El problema consiste en consultar y actualizar las calificaciones de cinco estudiantes sin perder la relación entre una persona y su nota. Un sistema pequeño puede ayudar a organizar un ejercicio académico, calcular resultados rápidamente y reducir errores al introducir datos.

La solución propuesta utiliza dos listas del mismo tamaño y orden. El elemento 0 de estudiantes es Ana y el elemento 0 de calificaciones es 85. Si se cambiara el orden de una sola lista, se podrían asignar notas a personas equivocadas. Por eso el programa modifica únicamente la nota en la posición seleccionada y no ordena las listas por separado.

Los nombres Ana, Luis, Carlos, Maria y Pedro y las notas 85, 67, 92, 74 y 58 son datos de ejemplo indicados en el enunciado. Las pruebas ejecutan ese conjunto y modificaciones de demostración. No se consultaron expedientes académicos ni bases de datos de estudiantes para obtenerlos.

La escala permitida va de 0 a 100 y admite enteros. El mínimo de aprobación es 70, una regla definida exclusivamente para este proyecto. Los datos permanecen en memoria: al cerrar se pierden las modificaciones y al iniciar otra sesión vuelven los valores originales. No se utilizan archivos de almacenamiento, bases de datos ni interfaces gráficas en la aplicación.

Desarrollo del programa

El archivo main.py contiene la aplicación y utiliza funciones sencillas. Primero se prepararon las listas y el menú; luego se ejecutaron sesiones con entradas controladas y se comprobaron las salidas. Los registros conservan tanto las entradas enviadas como el texto producido y el código de salida del proceso.

Python ejecuta el código. Visual Studio Code es el editor previsto para revisarlo en clase. La terminal de Windows permite iniciar el programa. Git sirve para registrar versiones y GitHub para alojar y compartir el repositorio. El acceso al perfil de GitHub se comprobó, pero el proyecto no se ha publicado. No se atribuye al estudiante el uso de herramientas que todavía debe revisar o ejecutar personalmente.

La opción 1 muestra nombres, notas e índices. La 2 consulta una posición. La 3 calcula el promedio; 4 y 5 muestran máximo y mínimo. La 6 muestra y cuenta aprobados y la 7 hace lo mismo con reprobados. La 8 cambia una nota y muestra el valor anterior y el nuevo. La 9 busca un nombre sin distinguir mayúsculas y quitando espacios externos. La 10 muestra el resumen actualizado. La opción 0 termina el programa.

leer_entero convierte la entrada mediante int. Si hay letras o un decimal, captura ValueError y solicita otro entero. Si el valor queda fuera del rango, muestra los límites y repite la pregunta. Un índice incorrecto no llega a utilizarse para acceder a la lista. Las interrupciones de entrada y de teclado también reciben un mensaje de cierre.

Explicación del código

1. Creación de los vectores

```
estudiantes = ["Ana", "Luis", "Carlos", "Maria", "Pedro"]
calificaciones = [85, 67, 92, 74, 58]
```

Cada lista almacena cinco elementos. El resultado esperado al mostrar ambas listas es Ana 85, Luis 67, Carlos 92, Maria 74 y Pedro 58. Esto aplica el almacenamiento de varios datos y la correspondencia por posición estudiada en el capítulo.

2. Acceso mediante índices

```
indice = leer_entero("Índice del estudiante (0 a 4): ", 0, 4)
print(f"{estudiantes[indice]}: {calificaciones[indice]} puntos")
```

Después de validar la posición se consulta el mismo índice en las dos listas. Al introducir 2 se muestra Carlos: 92 puntos. El índice permite recuperar una pareja de datos sin recorrer manualmente las posiciones anteriores.

3. Recorrido mediante ciclos

```
for indice in range(len(estudiantes)):
    print(f"{indice}. {estudiantes[indice]}: {calificaciones[indice]} puntos")
```

len obtiene cinco y range(5) genera los índices 0, 1, 2, 3 y 4. El ciclo repite la impresión para cada uno. El resultado son las cinco líneas del listado inicial. Se relacionan aquí el recorrido del vector y el acceso mediante subíndices.

4. Condiciones y conteo

```
aprobo = calificaciones[indice] >= MINIMO_APROBACION
if aprobo == aprobados:
    print(f"{estudiantes[indice]}: {calificaciones[indice]} puntos")
    cantidad = cantidad + 1
```

La comparación determina si una nota llega a 70. El parámetro aprobados indica qué grupo se quiere mostrar: True para aprobados y False para reprobados. Cada coincidencia aumenta el contador. Inicialmente hay tres aprobados y dos reprobados. Las condiciones permiten decidir qué elementos cumplen una regla; if, elif y else también seleccionan las operaciones del menú.

5. Cálculo del promedio

```
total = 0
for nota in calificaciones:
    total = total + nota
return total / len(calificaciones)
```

El acumulador empieza en cero y suma cada calificación. La suma inicial es 85 + 67 + 92 + 74 + 58 = 376; 376 / 5 = 75.2. El formato de impresión muestra 75.20. Se combinan recorrido y operaciones matemáticas. La función supone una lista no vacía; en esta aplicación siempre existen cinco notas.

6. Modificación

```
anterior = calificaciones[indice]
calificaciones[indice] = nueva
print(f"{estudiantes[indice]}: antes {anterior}, ahora {nueva} puntos")
```

Antes de este fragmento se validan el índice y la nueva nota. Para el índice 1 y el valor 80, la salida indica Luis: antes 67, ahora 80 puntos. La lista queda [85, 80, 92, 74, 58] y el promedio pasa a 77.80. La asignación sustituye un elemento y conserva su correspondencia con el nombre.

7. Búsqueda

```
for indice in range(len(estudiantes)):
    if estudiantes[indice].lower() == nombre.strip().lower():
        return indice
return -1
```

La función compara los nombres uno a uno. lower permite buscar Luis escribiendo luis, y strip elimina espacios al comienzo y al final de la entrada. Se devuelve 1 para Luis y -1 si no hay coincidencia; el menú comprueba -1 antes de acceder a las listas y muestra Estudiante no registrado. No se eliminan tildes: los datos iniciales contienen Maria sin tilde. La búsqueda aplica recorrido y comparación.

Repositorio de GitHub

Nombre preparado: sistema-calificaciones-python-capitulo-ix. La carpeta contiene el código principal, README, archivo de exclusiones de Git, pruebas, transcripciones y documentación. El código completo está disponible localmente en main.py.

El repositorio público fue creado en la cuenta mrcarlos1990 con autorización expresa del estudiante. GitHub confirmó la visibilidad pública y el enlace permite al profesor revisar los archivos sin solicitar permisos: https://github.com/mrcarlos1990/sistema-calificaciones-python-capitulo-ix.

Se publican el código completo, las pruebas, las transcripciones y la documentación. Las siete capturas auténticas de Visual Studio Code todavía deben obtenerse, revisarse e incorporarse. Por esa razón, el PDF continúa como versión para revisión y no se declara terminado el requisito de evidencias gráficas.

Evidencias de ejecución

Hay nueve registros TXT de ejecuciones auténticas en evidencias, uno por sesión de prueba. Cada registro contiene entradas, salida estándar, salida de errores y código de salida. resultados.json conserva los resultados comparados. Estos archivos permiten revisar y reproducir lo comprobado, pero no se presentan como capturas de pantalla.

Las figuras 1 a 7 quedan pendientes: código abierto en VS Code, vectores iniciales, menú, resultados académicos, modificación, validación y cierre. GUIA_CAPTURAS.md describe cómo obtener cada una en Windows e incluye el título y la explicación que deben acompañarla. La herramienta disponible no permite acceder a las aplicaciones nativas para tomarlas.

Como ejemplos de evidencia textual, ejecucion_02.txt registra Promedio general: 75.20; ejecucion_04.txt registra Luis: antes 67, ahora 80 puntos y Promedio general: 77.80; ejecucion_06.txt registra los mensajes de validación y la continuidad del programa. No se han creado imágenes vacías ni reconstrucciones gráficas de esas salidas.

Capturas revisadas e incorporadas: 0 de 7. El documento permanece pendiente de completar las figuras auténticas.

Casos de prueba

Se ejecutó python -m unittest discover -s pruebas -v con Python 3.12.14. Un método automatizado contiene nueve sesiones independientes. Cada sesión inicia el programa con las listas originales y termina con la opción 0. Los estados de la tabla se obtienen del archivo resultados.json generado por esa ejecución; no se asignan sin comprobar las salidas.

La modificación de Luis a 80 se comprueba dentro de una sola sesión. Las pruebas de aprobados y reprobados parten de los datos iniciales en otra sesión. La última sesión comprueba notas 0, 100 y el umbral exacto de 70. Los detalles íntegros, incluidas las entradas y las salidas completas, están en las transcripciones.

Prueba	Datos utilizados	Resultado esperado	Resultado obtenido	Estado
1. Visualización inicial	1; 0	Cinco parejas: Ana 85, Luis 67, Carlos 92, Maria 74, Pedro 58	Cinco parejas: Ana 85, Luis 67, Carlos 92, Maria 74, Pedro 58. Registro 01.	APROBADA
2. Promedio	3; 0	376 / 5 = 75.20	376 / 5 = 75.20. Registro 02.	APROBADA
3. Consulta por índice	2; índice 2; 0	Carlos: 92 puntos	Carlos: 92 puntos. Registro 03.	APROBADA
4. Modificación	8; índice 1; nota 80; 1; 3; 0	Luis: 67 a 80; listado con 80; promedio 77.80	Luis: 67 a 80; listado con 80; promedio 77.80. Registro 04.	APROBADA
5. Aprobados y reprobados	6; 7; 0	3 aprobados y 2 reprobados; nombres correspondientes	3 aprobados y 2 reprobados; nombres correspondientes. Registro 05.	APROBADA
6. Validación y continuidad	Letras, opción 99, índices 9/-1, notas -5/101/abc; luego valores válidos	Avisos, corrección y Ana 90; cierre normal	Avisos, corrección y Ana 90; cierre normal. Registro 06.	APROBADA
7. Búsqueda	lUiS con espacios; Sofia	Luis en índice 1; Sofia no registrado	Luis en índice 1; Sofia no registrado. Registro 07.	APROBADA
8. Extremos y resumen	10; 0	Promedio 75.20; máximo 92; mínimo 58; grupos 3/2	Promedio 75.20; máximo 92; mínimo 58; grupos 3/2. Registro 08.	APROBADA
9. Límites y aprobación en 70	Ana 0; Luis 100; Pedro 70; resumen	Promedio 67.20; 4 aprobados y 1 reprobado	Promedio 67.20; 4 aprobados y 1 reprobado. Registro 09.	APROBADA

Dificultades encontradas y soluciones

No se documentan dificultades personales del estudiante porque no fueron confirmadas. Durante la preparación se detectó una limitación verificable del entorno: no hay control de la interfaz nativa para capturar VS Code y la terminal. La solución disponible consiste en conservar las salidas reales y facilitar una guía para tomar las imágenes manualmente.

Espacio para revisión del estudiante: ¿Qué parte del ejercicio me resultó difícil? ¿Qué error observé realmente? ¿Cómo lo corregí? ¿Qué explicación puedo dar ahora? Completar estas respuestas únicamente con la experiencia real al revisar y ejecutar el programa.

Conclusión

El sistema muestra que una lista permite reunir varias calificaciones y procesarlas con pocas instrucciones. Los índices son importantes porque identifican las posiciones y mantienen la relación entre cada estudiante y su nota. Un vector facilita el recorrido de datos sin crear una variable por cada valor.

El ciclo for permite listar, sumar y buscar; while mantiene el menú en funcionamiento y permite corregir entradas. Las condiciones seleccionan las opciones y clasifican las notas según el mínimo de 70 definido para el ejercicio. Se aplicaron asignaciones, operaciones matemáticas, funciones, validación y presentación de resultados.

Este tipo de programa podría servir como base para organizar ejercicios de calificaciones u otras colecciones pequeñas. Su alcance actual es una sesión de consola con cinco estudiantes; no reemplaza un sistema académico con almacenamiento permanente. La guía de defensa permite revisar los conceptos antes de la evaluación. Las afirmaciones sobre aprendizaje personal deben ser confirmadas y adaptadas por el estudiante.

Fuentes consultadas

Python Software Foundation. Una introducción informal a Python, sección Listas. Documentación oficial. https://docs.python.org/es/3/tutorial/introduction.html. Consulta: 10 de octubre de 2026. Recurso utilizado para verificar listas, acceso por posición y modificación.

Python Software Foundation. Estructuras de datos. Tutorial oficial de Python. https://docs.python.org/es/3/tutorial/datastructures.html. Consulta: 10 de octubre de 2026. Referencia para las operaciones sobre listas.

Python Software Foundation. Más herramientas para control de flujo. Tutorial oficial de Python. https://docs.python.org/es/3/tutorial/controlflow.html. Consulta: 10 de octubre de 2026. Referencia para for, condiciones y control de ciclos.

Las explicaciones y los ejemplos del proyecto se redactaron para esta asignación. Las referencias respaldan el comportamiento del lenguaje y no se reproducen definiciones extensas de las páginas consultadas.